OCP GLOBAL SUMMIT 2026 • LATTICE / INTEL

无闪存启动与流式固件

Munir Ahmad（Lattice 首席计算架构师）
Alex Cervantes（Intel 云与 AI 系统架构师）

时长 18 分 29 秒

来源 youtube.com/watch?v=t845BOkcVcQ

无闪存启动与流式固件：面向安全可扩展基础设施的固件架构

> **整理说明**　本文是 OCP Global Summit 2026 一场 18 分 29 秒演讲的图文实录。体例为：每页幻灯片 → 该页停留期间的**完整中文译文**（逐句对应，不删减、不概括）→ 英文原句 → 图下一句小结。分节严格按幻灯片切换顺序，标题不含时间戳，时间信息集中在附录时间轴。

讲者

- **Munir Ahmad** — Chief Compute Architect, Lattice Semiconductor（首席计算架构师）
- **Alex Cervantes** — Cloud and AI Systems Architect, Intel（云与 AI 系统架构师）

这场演讲的位置　它不讨论某颗芯片，而是讨论一个运维层的结构性问题：服务器固件目前散落在每台机器的 SPI 闪存里，由 ODM/OEM 在出厂时烧录。两位讲者主张把固件从这个位置移走——要么在网络侧集中存放、按需流式送达，要么只留极小一段"早期固件"在本地作为信任锚，其余全部从外部获取。

关于自动字幕的更正　自动字幕在这类缩写密集的技术场次错误率很高，本文已按下表统一更正，正文中的数字与专有名词均经幻灯片原图核对。

字幕原文	更正为	说明
Lattis / Lattis Semiconductor	Lattice Semiconductor	讲者所属公司
Alex Arvantes	Alex Cervantes	讲者姓名
Mon	Munir	讲者自称
spy / espy / is by / eaf mode	SPI / eSPI	串行外设接口
PA / PJ / FJ / FPJ / PGA	FPGA	现场可编程门阵列
SRAMM	SRAM	静态随机存储器
DM / DM training	DMI / DMI training	直接媒体接口及其训练流程
Ali firmware / AI firmware	early firmware	早期固件
PO flash	PCH flash	平台控制器 Hub
flyless boot / flash wood	flash-less boot	无闪存启动
HP gen 111	HPE Gen11	惠普企业级服务器代次
OV obf cloud	OVHcloud	云服务商

字幕原文	更正为	说明
PLM	PLDM	平台级数据模型
CPSS	（保留原音）	讲者口播的容量数字，幻灯片未给出对应项

第一部分　问题

01　开场：把固件加载收回平台内部



Advancing Flash-less Boot in OCP Systems：A Streaming Firmware Architecture for Secure and Scalable Infrastructure，Munir Ahmad（Lattice）与 Alex Cervantes（Intel）

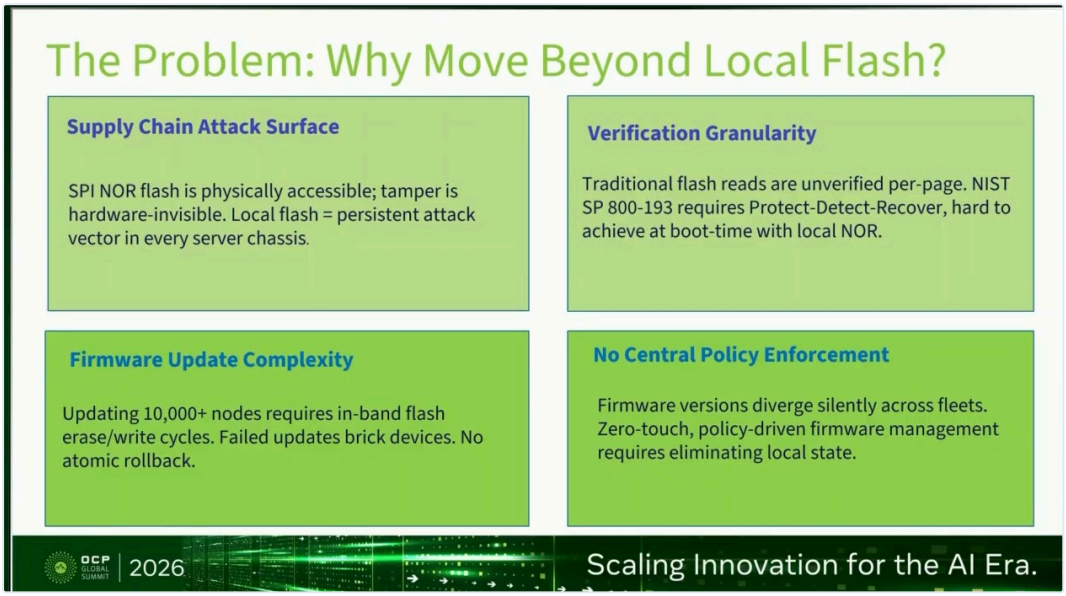
好，谢谢。这样可以吗？好。大家好。今天我们要讲的是流式启动（streaming boot）和无闪存启动（flash-less boot）这两个概念——过去几次我们已经在不同场合谈过一些。关于完全自治的系统层面需要做什么，已经有很多演讲讨论过。如果你希望在平台内部实现完整的控制，不依赖 ODM 或 OEM 来加载固件，那这就是我们要谈的。所以我们会讲一些我们一直在实验的东西，我们一直在把这些议题提出来。今天的讲者是我本人 Munir，来自 Lattice；以及 Alex Cervantes，来自 Intel。我们会从无闪存启动和流式启动的区别讲起，也就是为什么需要它、为什么不沿用现有的启动方案……

> Okay. Thank you. Is that okay? Yeah. Yeah. Hi. Uh today we're going to talk about uh the streaming boot flash boot concept that u we have been talking for several in a few times so far. Um you know it's a there's lot of talks about what needs to be done when you look at in a uh totally autonomous system level. If you look at a complete control within the platform itself itself not dependent on ODM or OEM to load the firmware. So we're going to talk about some of the things that we have been experimenting. We have been sending these issues on those things. So today u I'm presenting with myself Mon. I'm from Lattis and I'm Alex Arvantes. I'm from Intel. So yeah we're going to start with the

> differences on the flashless boot and streaming boot. So pretty much is why yeah why it's needed why we don't think on a different boot solution that the current solution that we have but pretty much is we can talk about the different for uh

> **小结**　这场演讲要争的不是"启动快几秒"，而是固件的管辖权：把固件从每台机器的闪存芯片里抽出来，让平台自己决定加载什么、从哪里加载。

02　四个必须离开本地闪存的理由



The Problem: Why Move Beyond Local Flash? 四宫格：供应链攻击面、验证粒度、固件更新复杂度、缺乏集中策略

……当前方案的差异。基本上我们可以谈四个向量。第一个是**供应链攻击面**：当我们把所有固件、所有配方都放在节点或计算系统上，在它们被安装进数据中心时就已经在那里了，我们等于把这个环节从芯片上移除。另一个是**验证粒度**：在传统闪存上做验证没那么容易，而在这种方式下，我们可以验证来自流式传输器或无闪存启动的镜像，取决于我们把镜像放在哪里。第三个——这也是我个人最喜欢的——是**固件更新复杂度**。如果你有过给一个集群里若干节点做更新的经验，那需要花时间、花精力，主要是停机时间，才能正确地重新烧录、让每一个组件都保持最新，而这些停机时间并不一定是你想要的。有时候它会失败，有时候一个节点会在过程中途挂掉，这又增加了继续运行的顾虑和挑战。最后一个是**没有集中策略**：最终的想法是让所有节点都指向同一个地方，从而拿到硬件版本、固件配方等等，而不需要直接介入任何一台设备，因为一切都集中化了。

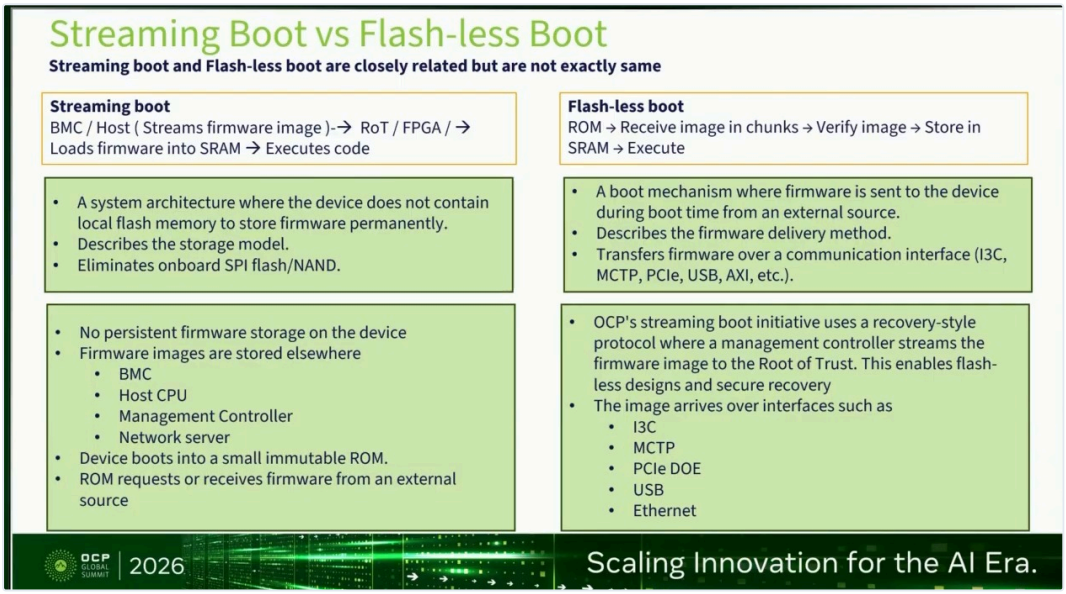
> different boot solution that the current solution that we have but pretty much is we can talk about the different for uh vectors uh one is like the supply chain uh attack surfaces we can have a control environment when we have all the firmware all the recipe and the nodes or the computes systems will have that and all when they are installed on the data center. So we remove that from the chip and etc. Another thing is the verification granularity. So on the traditional flashes the verification is not as easy to be done. In this way we can verify the image from from the wireless streamer or flagless boot depending on where we had it. And the neck which is like kind of my favorite on the firmware update complexity. H if you have experience on trying to update several nodes on a cluster uh it takes sometimes and takes an effort mainly the downtime in order to properly reflash and have every ingredient up to date and it's just downtimes that are not necessarily desirable. Sometimes it fails. Sometimes a node may break in the middle of the process which increases the the concerns and the challenges to continue operating. And the last is like no central policy like yeah at the end the idea is to have pretty much all the nodes uh going into the same place in order to reach or have the hware

> version fware recipe etc without intervention directly on any of the devices because pretty much everything is like a centralized.

> **小结**　幻灯片把四个理由并列，但它们的量级并不相同：前三项是技术缺陷，第四项是运维模型缺陷——只要固件还留在本地，"零接触"就无从谈起。

第二部分　两条路线

03　流式启动与无闪存启动不是一回事



Streaming Boot vs Flash-less Boot：左侧流式启动（BMC/Host 流式传输 → RoT/FPGA → 载入 SRAM → 执行），右侧无闪存启动（ROM → 分块接收 → 验证 → 存入 SRAM → 执行）

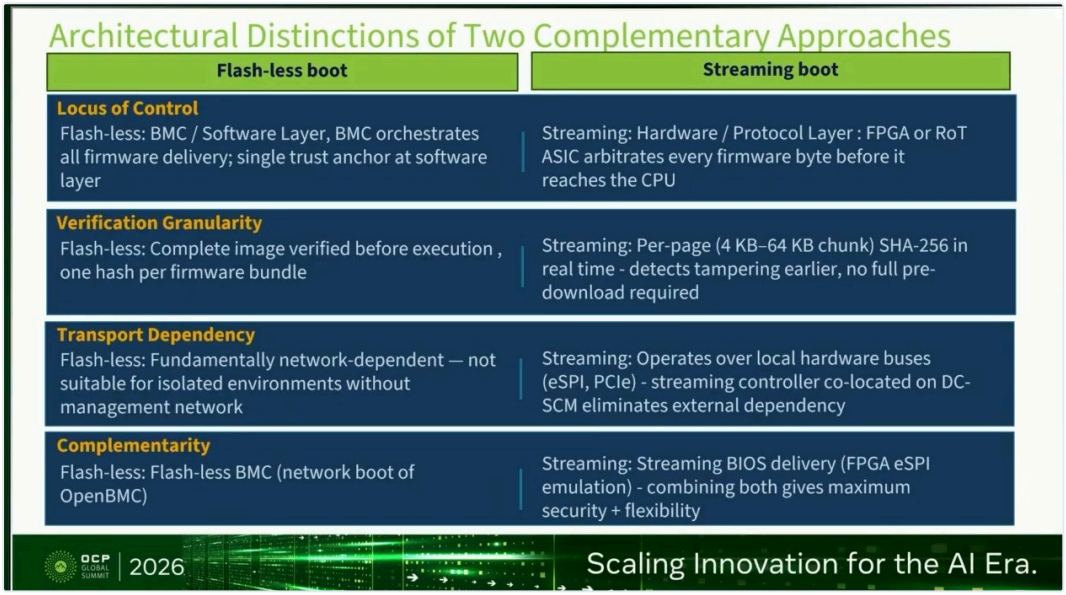
那我们再深入一点，讲这两种方式的差异。左边是流式启动。流式启动本质上是通过网络流把固件提供给节点——你有基础设施层，所有固件都存放在那里；然后系统自己从一段不可变的 ROM 启动。系统会知道——比如说 BMC 或 FPGA 会知道——去哪里取数据，然后开始抓取数据、把数据加载到系统的 SRAM 里。之后这些数据可以被转换，并以某种状态呈现给 CPU。所以在这个流式传输的部分，系统上完全不需要有闪存器件，只需要知道从哪里拿到镜像、从哪里拿到设备的固件。右边是无闪存启动，它很相似，但主要区别在于：无闪存启动里我们在系统上保留一小部分，用来放一段"早期固件启动"，由它告诉系统去哪里找其余的固件。早期固件只有很少的指令，说明从哪里取东西；其余的固件才包含真正的 BIOS 和可执行文件等等。

> So going into a little bit more on the two approaches the differences. So on the on the left we have the streaming boot. Pretty much the streaming boot is a way of providing the firmware to the node via network stream like um uh you have your infrastructure where you have all your your firmware residing on and then on the system the system will start by itself from a immutable ROM. So the it system will know uh let's say the BMC or the PA will know where to go and get the data and it start like a grabbing that data and start loading the data in the SRAMM on the on the SRAMM of the system. So that can be then translated and and present as stated to the CPU. So pretty much on this streaming portion there is no need to have a flash device on the systems. Pretty much just a way to know where where to get h where to

> get the the images where to get the the firmware for the devices. On the right side we have the flashless boot which is quite similar but the main difference is that on the flashless boot we have something a small portion in the system small portion to have like an early firmware boot that's called in order to kind of tell the system where to look for the rest of the firmware which uh which is the the other portion like early firmware only has minor minor instruction on where where to get stuff and then the rest of the fware contains the actual BIOS and executables and etc.

> **小结**　两个名字容易被当成同义词，差别其实只在"那段最小固件放在哪"：流式启动靠不可变 ROM 起步，无闪存启动靠本地一小片闪存里的早期固件起步。

04　两条路线的架构分野



Architectural Distinctions of Two Complementary Approaches：控制点、验证粒度、传输依赖、互补性四行对照

继续讲两者在控制点上的差异。无闪存启动这条路上，我们有一个 BMC，它是主要的驱动者，负责编排从哪里、以什么方式取信息。而流式启动这条路，我们有一个硬件部分，由它告诉你去哪里、去哪里取、从哪里开始拿到这些细节、拿到所需的固件。再看验证粒度：无闪存启动是整份镜像下载完再校验，然后提供给设备；相比之下，流式启动是按页、按块、按 chunk 取数据并校验，之后才转交给 CPU。再看传输依赖：无闪存启动依赖网络，它依靠网络路径来获取数据，这是它的前提条件；流式启动类似，但还包括内部通路，也就是早期固件与其余固件之间如何交互、如何互补。无闪存启动里，BMC 是主设备；流式启动里则是一个 FPGA，它在做某种 API 的模拟——下一张图会更清楚。

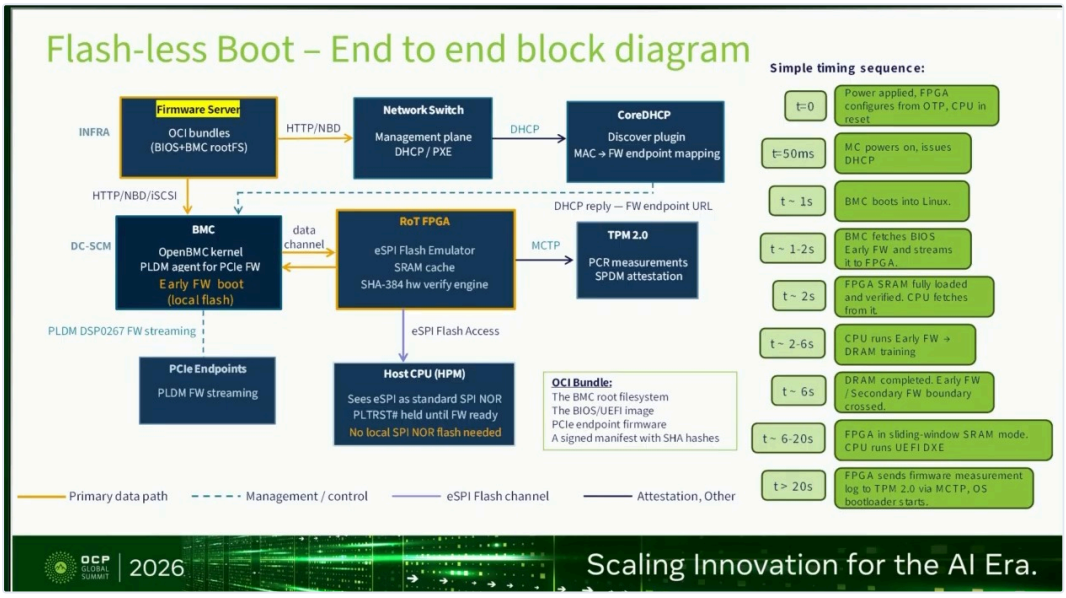
> So continuing with the kind of like the differences on both uh on the lock locus of control for the different for the both alternatives pretty much on the fly boot we have a BMC which is like the main main driver or who is is is orchestrating how or where to get the information and on the streaming boot is like a we have a hardware portion that tells where to go, where to fetch, where to start h getting all that details, all that uh the fware uh of the fware needed on the verification granularity. Yeah. The on the flashes boot the completed match is like a download and check and then provided to the device. In comparison with the streaming boot is like a per page or per chunks per blocks where the data is been being fetched or being uh obtained and being checked. After that it's that is transferred to the to the CPU on the transport dependency uh pretty much on the flashes boot option it is depending on the network erh it's it's as it relies on the network path to obtain the the the data it's it's a requirement for that and on the streaming boot it's like a similarly but also including that the internal patch is where the interaction on the different early and and and the rest of

> the fware behaves for the complimentary. Yeah, the flyless boot is a BMC like a BMC main main device or or or ingredient and for the streaming boot pretty much is like an FPGA or it's like a FPJ trying to do kind of like a emulation of the API but I think it's clearer on the on the next.

> **小结**　控制点从"软件（BMC 编排）"移到"硬件（FPGA/RoT 逐字节仲裁）"，验证粒度也随之从整包哈希变成 4–64 KB 分块哈希——这是两条路线真正的分水岭。

第三部分　端到端流程

05　无闪存启动的端到端流程



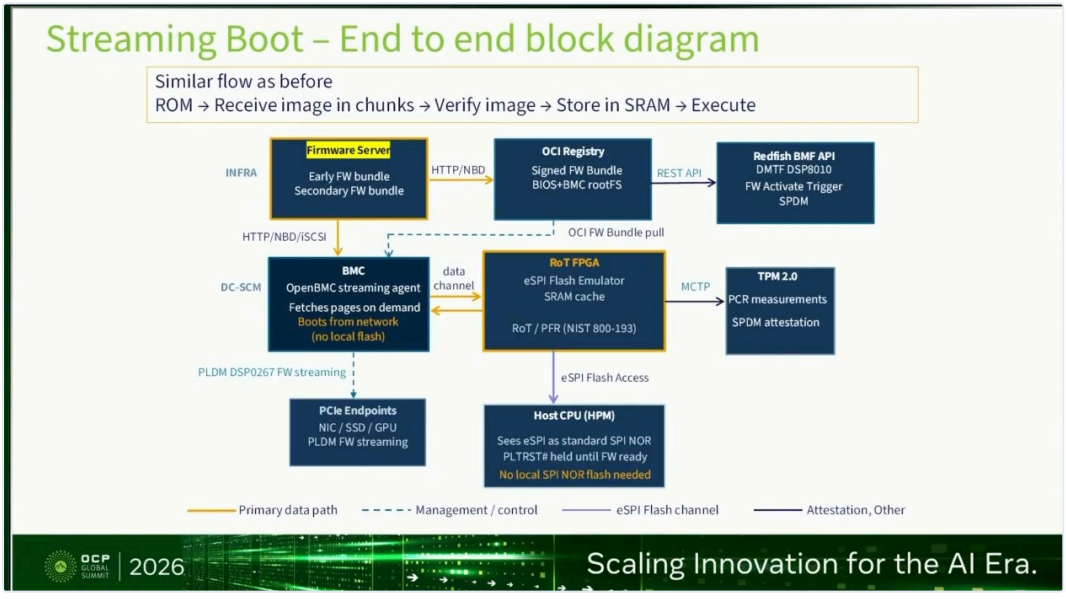
Flash-less Boot – End to end block diagram：基础设施层（Firmware Server、Network Switch、CoreDHCP）、DC-SCM 层（BMC、RoT FPGA、TPM 2.0）、PCIe 端点与 Host CPU，右侧为 t=0 到 t>20s 的时序

接下来这张是无闪存启动实现的框图，我们可以看到参与的各个设备。顶部是基础设施部分，镜像存放在那里，也就是数据中心的控制网络侧。下面是机箱部分。左边的设备是 BMC，它负责与基础设施服务器通信。启动过程中，BMC 启动电源时序，并加载最低限度的数据，用来知道该做什么、去哪里取其余组件——也就是早期固件。早期固件存放在本地一小片闪存上，只是为了让系统有一个最低限度的方向。之后 BMC 自己启动，检查它的早期固件，然后它就知道该去基础设施取其余的固件。取回的数据被传输或共享给 FPGA，由 FPGA 按粒度分块校验，然后做 TPM 上的认证等等，最后送到 CPU。这里与常规启动流程、常规服务器的关键差异在于：FPGA 是主要的区别点。CPU 会尝试通过 eSPI 或类似模式启动，而 FPGA 承担全部处理，把数据提供给 CPU，CPU 并不知道数据究竟从哪里来。FPGA 会模拟 SPI 设备的行为，模拟 SPI 设备如何把数据准备好交给 CPU。右边是一个简单的时序图，展示主要事件：BMC 开始取早期固件，然后启动，准备好之后提供给 FPGA，FPGA 再提供给 CPU，如此继续。这只是个粗略示意，给大家一个视觉上的概念。下一张是流式启动，与上一张很相似，区别在于现在整份镜像被加载到 BMC 里……

> the next. So this is a block diagram of the flashless boot implementation. We can see the different devices that are part of this. On the top we have uh the infrastructure portion of of of things where pretty much the image is stored or the networking the control networking side of the data center. And below that we have the chassis portion and the the device on the left is the BMC which is the one that is communicating to the to the infrastructure server. From that on the boot process the BMC starts starts the the the power sequence let's say and it loads the minimum requirement data as to to know what to do to know where to get the rest of the ingredients which is the early firmware. That early firmware is stored on a local flash a small local flash just to have a minimum minimum idea where where to do that. After that the BMC erh boots by itself, checks its early firmware and then it knows and goes to the infrastructure to get the rest of the of the firmware. So that is then trans transport or shared to the FPA which is checking is checking like the grularity in chunks and then for the stationation on the TPM etc. then at the end it goes to the CPU. The thing here or the main difference with a regular bootflow or a regular server without this is that in this case the FPGA is like the main main how to say differentiator as the CPU will try to boot via espy or kind of like a eaf mode and the PGA will do all the handling at all the handling to provide the data to the CPU and the CPU will will not know exactly where it came but The FPJ will emulate or is simulating how the the any is spy device erh how how the isy device provides how the spy device has the data and and ready for the CPU. And on the right we have just a a simple timing diagram sequence on the different main main stuff that are happening like BMC starting to fetch the early firmware then it boots then when it's ready it has that provided to the PA then the PGA provide to the CPU and then yeah it continues. It's a lot a rough idea just to to have uh something visually to understand. And next is the streaming boot which is quite similar to the previous in the difference that now uh the the image is load like at the

小结　这套流程的核心技巧是"欺骗"：CPU 以为自己在读一颗本地 SPI NOR 闪存，实际上数据来自网络，中间由 FPGA 做 eSPI 仿真。时序图里 t≈1–2 s 取回 BIOS 这一段是关键路径。

06　流式启动的端到端流程



Streaming Boot – End to end block diagram：新增 OCI Registry 与 Redfish BMF API，BMC 变为 OpenBMC streaming agent，从网络按需取页、无本地闪存

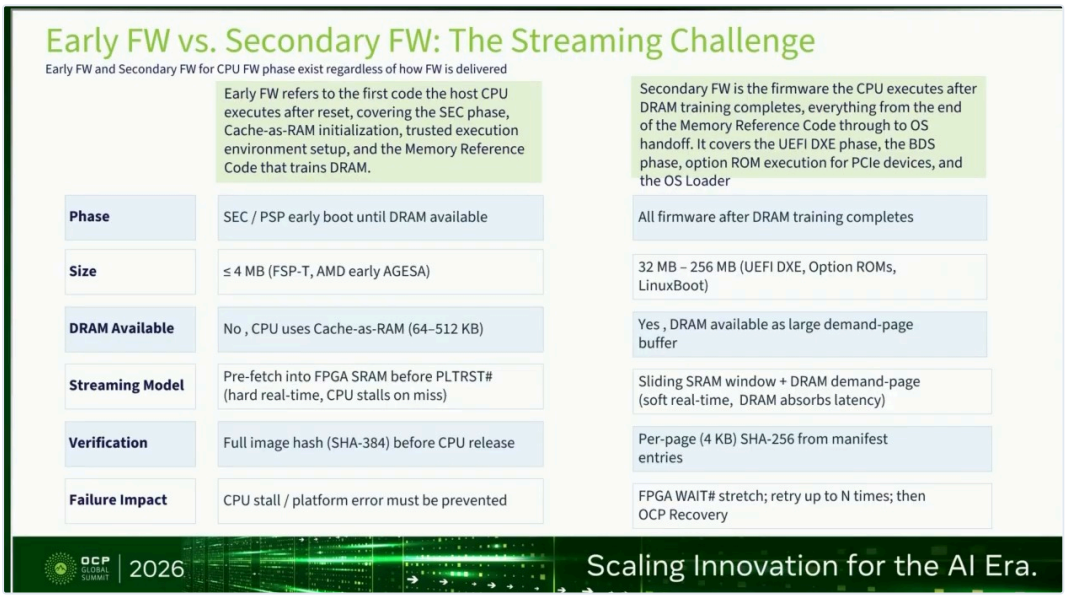
……整份镜像加载到 BMC 里，然后 BMC 把它转给 FPGA，FPGA 再以同样的方式把它呈现给 CPU，就像它是一个 eSPI 类型的设备。所以……我接下来翻到下一页。谢谢 Alex。我认为在高层面上，两者并没有区别——好吧，是有一些区别：流式启动看的是"你如何交付镜像"的机制，而无闪存启动看的是"你把这份初始镜像放在哪里"的机制……

> whole image in the BMC and then the BMC translate that to the PJ the PJ present that to the CPU in the same sense as an is by type of device. So so I yeah so I think let me go to the next slide. Um so I think u thanks thanks u Alex I think I think in a high level you know there is no difference between a you know well there's some difference the streaming boot is basically uh looking at the mechanism how you deliver the image and whereas flashless boot is basically a mechanism

小结　这一页比上一页多了两件基础设施组件——OCI Registry 存签名固件包、Redfish BMF API 做触发与激活，BMC 则从"取整包"变成"按需取页"的 streaming agent。

第四部分　工程难点

07　早期固件与次级固件：真正的难点



Early FW vs. Secondary FW: The Streaming Challenge：阶段、大小、DRAM 可用性、流式模型、验证方式、失败影响六行对照

……也就是说，在高层面上……我们做了一些实验，就像 Alex 展示的那些时序，因为这是重要的部分。但整件事最难的地方在于：无论你选哪种机制，都必须有一段早期固件和一段次级固件，并且要让它们以某种方式协同工作。我说的早期固件是指：一个空白的 CPU 没有任何办法去下载大容量的东西。早期固件的概念就是让 CPU 准备好，从 DMI 或其他阶段接收镜像——那些阶段可以处理巨量数据。所以理想情况下早期固件应该非常小，你要让它尽可能简单、尽可能小。它应该包含 DMI 训练时序以及所有必须在那里完成的事情。另外还有一个附加条件：你可能希望在那里放一份安全检查清单，因为一旦你验证了早期固件，早期固件就可以去验证次级固件——如果你选择这种方式的话。这是另一个可以做的附加项。所以拥有早期固件大概是最难的部分，因为早期固件有非常非常紧的时间约束，你必须确保它在特定时间内响应，这是 CPU 的要求。如果你看左边，我们建议早期固件应该小于——我记得他们给出的数字，我看到 CPSS 大约是 6 MB，但我们看到的情况是接近 4 MB。而且早期固件也需要某种验证，因为你是把早期固件当作次级固件的安全基础。次级固件

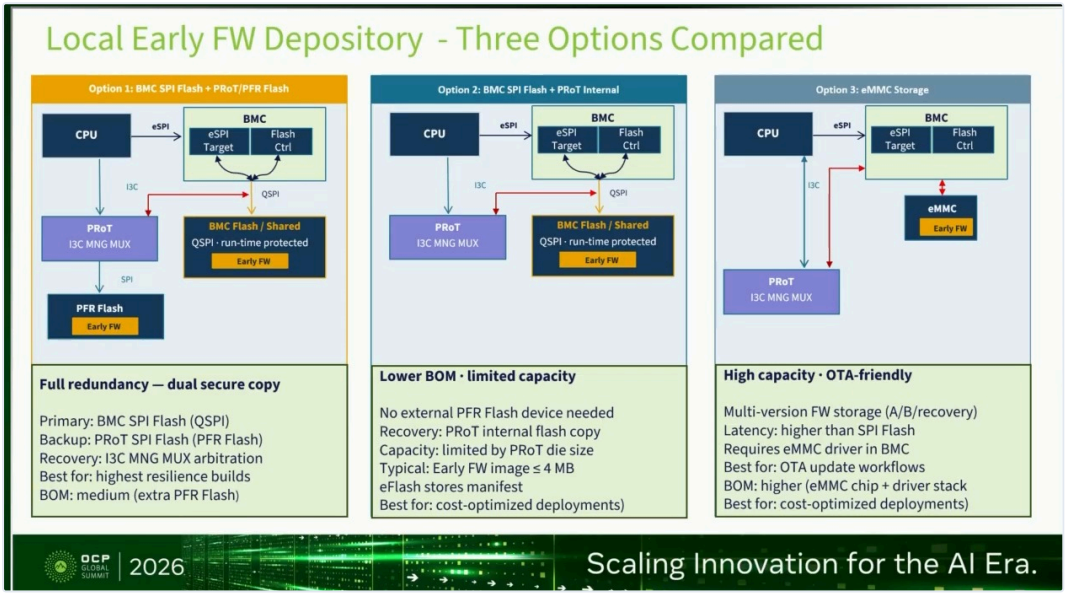
基本上是：当 DMI 训练完成，CPU 可以从 DMI 的角度放入镜像，那在次级固件阶段会快得多，然后你也可以在那里做某种安全检查。所以早期固件最大的挑战是：在早期阶段几乎没有延迟容忍度，因为 CPU 会立即开始取数据，你在取数过程中看到的任何延迟都可能让 CPU 挂起、根本无法启动——这是一个非常关键的阶段，因为 CPU 就是这样工作的。我们用 Intel CPU 做过一些实验，这是我们发现的。现在我不认为有——有一些实验性的方式可以让 PC 工作，但我不认为目前有任何生产级的工作完成。另外早期固件没有 DMI 缓冲，所以我认为我们唯一的选择是用 APGS RAM 来存放这 4 MB 数据作为早期固件。我们试图解决的关键问题之一是：由于复位、由于这类信号的时序要求，你希望早期固件是一份非常顺序化的镜像。所以这不只是固件的问题，eSPI 通道上还有别的信号来回走。这必须是高度可预测的，而今天它并不可预测，这是我们仍然面对的挑战之一。我不是说它顺序化，我是说这是另一个我们面对的挑战。我想这里有三条信息——我知道我时间不多了，但我要展示三条信息，说明早期固件可以待在哪里。在很高的层面上，我们可以让早期固件待在 BMC 里……抱歉，好，谢谢。这里有三个匹配项：你可以让 BMC 的 SPI 闪存保存固件，次级可以放在 PCH 闪存里保存早期固件；你也可以把所有东西都放进 PCH；或者如果你想要不止一份固件镜像——如果你有分区系统之类的东西——你可以增加只与 PCH 通信的额外 eMMC……

> where you going to put this initial image so that's at a high level very and we did some experimentation as as Alex was showing of some of the timing sequence because this is the important but the hardest part of this whole thing is to have you know no matter what mechanism you go there is there is to is some kind of early firmware and there's going to be some kind of secondary firmware that you need to have an some way of somewhat working right what I mean by early firmware is that in in a CPU without anything it doesn't have any any way to download anything that is high volume right so the early firmware is a concept of it is that the early firmware will makes the CPU ready for taking images from the DM or some other stages where basically it can do huge volume of data right so ideally the early is supposed to be very small one. U you know you want to keep it as simple as possible as small as possible. It should have the DM training sequencing and all those thing that needs to be done there. Uh and also I think there is also an another caveat there that you may want to have some kind of security checklist there because once you validate the AI firmware then Ali firmware can excuse me can validate the secondary firmware if you want to do that way. That's another caveat that you can do. So having the early formula is the is probably the toughest one because there's a lot of things that early formula is a very very tight time con you know time constraint right you have to make sure that it responds in a certain time because of that you know uh the CPU requirement right it has to be like um you know so [clears throat] that's the one if you look at the left side we're suggesting that Ali firmware can be should be less than you know whatever the size I think they have I saw CPSS about 6 megaby but I think uh We we saw something like coming at 4 megabyte u and also that there there'll be needs to be some verification ali firmware because you're basing Ali format to be the uh security base for the secondary firmware right and the secondary firmware will basically be when the DM train is completed CPU can put the image from the DM perspective so that will be much faster in the secondary uh firmware right and then you can do also the some kind of security check there so biggest challenge the early firmware is that there is pretty much zero latency tolerance at the early stage because CPU start immediately fetching any kind of delay you see in the fetching can basically put the CPU in the hung no boot up is possible right that's a very critical stage because that how the CPU works like we had some experimentation done with Intel CPU and that's something that we found out that uh and now right now I don't think there is a you know there is some experimental way of PC working but I don't think there is any production level work that has been done so far yet u also the early farmer there no DM buffer. So you know I think the only option that we used is APGS RAM basically to hold this you know 4 megab data as an R firmware and the one of the key problem that we are trying to solve is that because that you know because of the reset because of the timing requirement of this kind of signals you want to have a very sequential image of the early firmware right so it's it's not only firmware because but there is some also other kind of signals going back and forth on east by channels of Intel has like some kind of um so anyway I the so this has to be very predictable today it's not predictable so this is one of the challenges we are still facing um it's not say sequential I'm saying so that's another challenge we're facing a couple I think three message here I know I have a little bit time but I just going to show three messages where the early

> farmer going to stay I think in a very high level we can have early farming state in the BMC you know you know BMC inside oh sorry okay yeah thank you yeah so there's three matches here so you can have BMC spy flash can hold the firmware And the secondary you can have the PO flash holding the Ali firmware. You can also just make uh everything inside the PO or if you want to do more than one firmware image. If you have like partition system or anything like that, you can add additional eMMC that only

> **小结**　真正的工程瓶颈不在"从网络取固件"，而在取回之前的几秒钟：早期固件要小于 4 MB、要能在零延迟容忍下运行、要能在没有 DMI 缓冲的情况下驻留，而今天它还做不到完全可预测。

08　早期固件放哪里：三种方案



Local Early FW Depository - Three Options Compared：方案一 BMC SPI Flash + PRoT/PFR Flash（双份安全副本）、方案二 BMC SPI Flash + PRoT 内部（低 BOM）、方案三 eMMC 存储（高容量、OTA 友好）

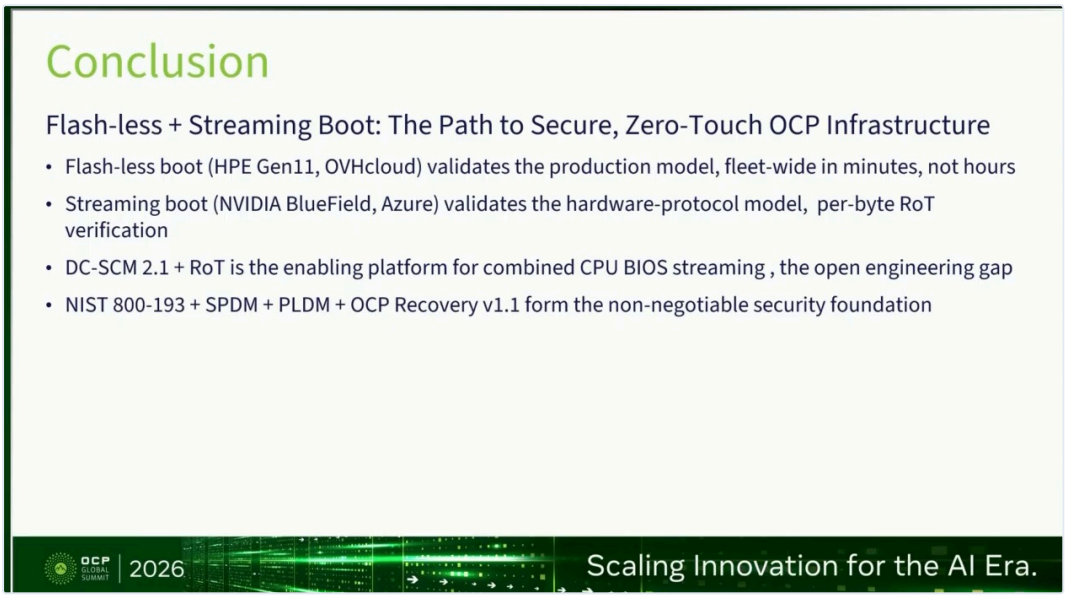
……只与 PCH 通信。这样就没有安全漏洞，可以这么做。这就是我们做的三个提案。去年 OCP 我们实现了其中一个，与 Intel 一起做了概念验证。但目前这只是我们这边的提案。我知道时间很短，所以我讲快一点……我想我还剩一分钟。

> talk to the PO. So there is no security hack there. It can be like that. Right? So there's these three proposals we did. We implemented one of them in last year OCP just to show the proof of concept with Intel. Uh but this is just a proposal from our side at this time. uh I know time is pretty short so I'm going to go quickly I [clears throat] think

> **小结**　三个方案对应三种取舍：双份闪存换最高韧性、PRoT 内部换最低 BOM、eMMC 换最大容量与 OTA 友好——没有哪一个在所有维度上都赢，选择取决于机队更怕"变砖"还是更怕"多一颗料"。

第五部分　结论与行动

09　结论：部署发生在需要的地方



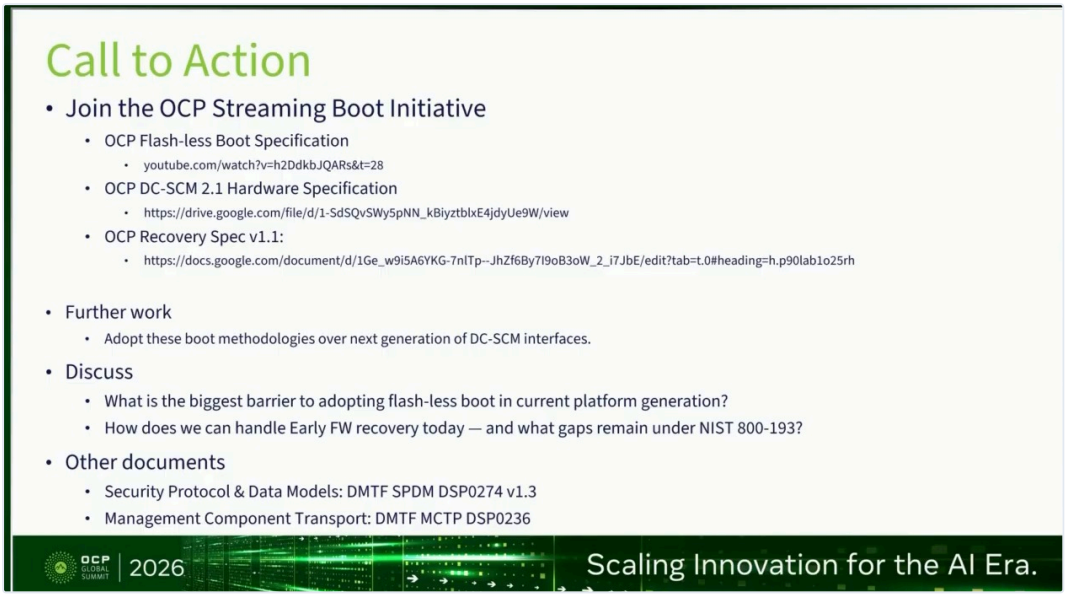
Conclusion：无闪存启动（HPE Gen11、OVHcloud）验证了生产模型，流式启动（NVIDIA BlueField、Azure）验证了硬件协议模型

……所以结论是：看看我们今天做的所有事情，依赖太多了。你有节点级、有多层 L1 到 L2、L12 级别的部署，而流式启动或无闪存启动解决的正是"在需要的地方部署"——你不需要保留多个版本的这些东西。所以无闪存启动，我认为 DC-SCM 和信任根可以在其中扮演非常重要的角色。然后，安全标准必须成为锚点，这一点非常重要。还有一些用例——你已经看到一些例子，比如 HPE Gen11，大约一年前他们展示的只有 BMC 镜像，还没到 CPU 启动那一侧，但他们展示了某种无闪存启动。还有 OVHcloud 也展示了类似的东西。我们做的其中一个演示，是从 CPU 侧做基于 eSPI 的启动，属于无闪存启动而不是流式启动——你可以去看看。然后是行动号召，基本上我们一直在推动，我们一直在和 OCP 合作，我们刚开设了一个新的工作组，这是定义启动结构的基本要求之一……

> it's one minute so the conclusion is that uh if you look at that everything that we're doing today is that uh there's so much dependency here right you have node level there's multiple L1 to L2 L L12 level of all the deployment and this is what what streaming boot or flash wood is solving is the deployment at the point of need right you don't need to have deployment keeping multiple versions of this thing so so flashless boot um I think the DCSM and root trust can play a very important role here. Um and then uh security standards must be an anchor. I think that's something is very important and there is some you know use cases some uh use not use cases some examples you have seen like HP gen 111 and they showed about a year ago they showed only the BMC you know image also not the boot CPU side but they showed some kind of u you know flashless boot there. as also uh OV obf cloud they showed something like that we had one of the demo that you're showing a espy based boot from the CPU at a flashless not the flash streaming boot so you're showing that feel free to check it on uh and then the so call to action is basically we have been asking uh you know we have been working with OCP there's a new work team that we just opened uh that's one of the

> **小结**　结论页最值得注意的是两条验证路径的分工：无闪存启动已被 HPE Gen11 与 OVHcloud 在生产机队验证（分钟级而非小时级），流式启动则被 NVIDIA BlueField 与 Azure 验证在硬件协议层。

10　行动号召与问答



Call to Action：加入 OCP Streaming Boot Initiative，列出 Flash-less Boot 规范、DC-SCM 2.1 硬件规范、Recovery Spec v1.1 三个文档链接，以及待讨论问题

……定义启动结构的基本要求。说到机架级——今天的机架级意味着你的工作负载由多个机架组合而成，所以你必须让它们全部版本一致。那么唯一的选择就是网络启动，或者某种闪存启动之类的方式。所以我们有一个开放的工作组，欢迎加入。我这里有一些……顺便说一句，这个链接是坏的。OCP 最近改了 G Drive 上那些东西的位置，所以我和 Alex 写的时候链接是好的，现在不通了。但这是我们需要处理的事。我想今天的核心信息就是：我们需要在流式启动、或者说无闪存启动上一起合作，这两者非常接近。时间到了，有问题请提。

提问者：我的问题是，PLDM 是一个 SRAM 占用很大的应用，这件事能做成吗？能不能不用 PLDM 就实现其中一种？

Munir Ahmad：所以 PLDM 基本上是在你要交付格式的时候用的，对吧？你是想通过 PLDM 来交付格式。如果我不想在两端的对等体上都跑 PLDM 快照……

提问者：通常在这种启动的情形下，你谈到的 PLDM 是在做格式更新的时候用的，但那意味着设备上已经有格式了。

Munir Ahmad：对。对。而在这种情况下你什么都没有，对吧？你没有任何固件，那里只有一些微码和 CPU 或 BMC。所以我认为一旦它启动起来，你随时可以用 PLDM 去更新固件，如果你想这么做的话。但我认为这里的整个想法是：一旦你把那些东西牵扯进来……我可以说，如果你通过 PLDM 有一个内部源，你可以做，但那是用于格式更新的，不是从零开始启动。

提问者：今天的 SPI 方案与无闪存方案之间，预期有速度差异吗？

Munir Ahmad：会更慢。我会说它显然会更慢，因为基于 SPI 的……对。

提问者：明显的秒级差异?

Munir Ahmad：我认为这只是加载的问题。对，我们在 PC 层面看到几乎翻倍。但这是我们需要做的优化，因为有很多信号——CPU 不只是取固件，同时还有大量握手发生。而这些握手我们今天必须去模拟，因为我们还不完全理解整个流程。因为 Intel 或 AMD 有大量遗留接口。所以我们的实验显示几乎是两倍，但我们正在努力。

提问者：好，我会……我肯定会采纳这个。

Munir Ahmad：请讲。谢谢各位。

> fundamental requirements to define the boot structure and when you talk about the rack level right now The rack level means right now today your workload comprises multiple racks together. So you need to have all those have the same version. So the only option is to have a network boot or something flash boot or something. So we have an open work team. Feel free to join there. Um I have some I think this will be the available. So I have some the link is not working by the way. The OCP recently changed their location of G drive on those things. So when we yeah wrote out myself and Alec that the link was working but it's not working. uh but yeah that's something uh we need to uh but uh I think this is what the message is today that we need to work together on the streaming boot because um or flashless boot and this is very close right um so any question I know my time is up but any question please so so my question is uh PLDM being a SRAM heavy application can can this be done can can one of these be achieved without using PLDM. So PLDM is basically when you want to deliver the format. That's what you're saying, right? You want to deliver the format through the PLDM. If I don't want to run the PLDM snapshot in both the peers typically when you have this situation of the boot up that uh you know with the C you were talking about PLM when you do a format update but that means that you already have a format available on this on the device. Yeah. Right. In this situation you don't have anything there. Right. you don't have any firmware nothing there that is just some microode and the CPU or BMC right so yeah I think once you boot it up then you can always use PLDM right once you boot it up you can always PLDM and something to update the firmware if you want to do that but I think the whole idea here is the moment you involve all those kind of you know things then now you're um I can I can do it I think if you have an inter source through the PLM you can do it yeah but that's for the format update not for the from the starting from zero to up Is there any expected speed differences between today's spy based versus flashes? It'll be slower. I would say that it will be slower obviously because spy based is you know it's yeah significantly seconds we I think it is just kind of the loading right yeah we have seen on the PC level it's almost doubled uh but that's something optimization that we need to do because lot of signals that um because CPU doesn't just take the firmware they have a lot of handshaking happens in the same time right and those handshake we have to simulate today uh because we don't [clears throat] understand the full process yet right you know because there's you know there's a lot of legacy you know interfaces that Intel has or AMD has. So we have been I think our experimental shows is almost double but I think we're trying to do it. Yeah. Okay. I will I will uh I will definitely take the Yeah, please. Thank you guys. Thank you.

> **小结**　问答里最有价值的一句是"几乎翻倍"：无闪存启动目前比本地 SPI 启动慢近一倍，原因是 CPU 的握手时序需要被 FPGA 逐条模拟，而这套遗留时序尚未被完整理解。这决定了它短期内只会先落在新建的超大规模机队上。

附录

附录 A　数据核对表

项目	数值 / 结论	来源	备注
演讲时长	18 分 29 秒（1108.8 s）	视频元数据	—
早期固件建议大小	≤ 4 MB	幻灯片 07	讲者口播称"看到 CPSS 约 6 MB，我们实测接近 4 MB"
次级固件大小	32 MB – 256 MB	幻灯片 07	UEFI DXE、Option ROM、LinuxBoot
DRAM 可用性（早期）	否，CPU 使用 Cache-as-RAM（64–512 KB）	幻灯片 07	—
早期固件验证	释放 CPU 前做整镜像哈希（SHA-384）	幻灯片 07	—
次级固件验证	每页（4 KB）SHA-256，来自 manifest 条目	幻灯片 07	—
流式验证分块	4 KB – 64 KB chunk，实时 SHA-256	幻灯片 04	无闪存启动为整包单哈希
无闪存启动时序	t≈1–2 s 取回 BIOS 早期固件并流入 FPGA	幻灯片 05	t≈6 s DRAM 训练完成
启动耗时差异	约为本地 SPI 的 2 倍	口播（问答）	幻灯片未给出该数字
生产验证案例	HPE Gen11、OVHcloud（无闪存）；NVIDIA BlueField、Azure（流式）	幻灯片 09	—
安全基线	NIST 800-193 + SPDM + PLDM + OCP Recovery v1.1	幻灯片 09	—
现有规范	OCP Flash-less Boot Spec、DC-SCM 2.1、Recovery Spec v1.1	幻灯片 10	讲者现场说明链接已失效

附录 B　时间轴

时间	内容
00:00	开场，介绍两位讲者与议题
00:53	四个理由：供应链攻击面、验证粒度、更新复杂度、缺乏集中策略
02:43	流式启动与无闪存启动的定义差异
04:31	架构对照：控制点、验证粒度、传输依赖、互补性
06:30	无闪存启动端到端框图与时序
09:14	流式启动端到端框图
09:46	早期固件与次级固件：大小、DRAM、验证、失败影响
13:31	早期固件存放的三个方案
14:05	结论：DC-SCM 与信任根、安全标准作为锚点
15:19	行动号召、工作组、文档链接失效说明
16:21	问答：PLDM 能否省略、SPI 与无闪存的速度差
18:20	结束

附录 C　术语表

缩写	全称	说明
SPI / eSPI	Serial Peripheral Interface / enhanced SPI	CPU 读取固件的传统总线；eSPI 为其增强版
BMC	Baseboard Management Controller	基板管理控制器，平台的带外管理核心
RoT	Root of Trust	信任根，负责度量与校验
FPGA	Field Programmable Gate Array	此处承担 eSPI Flash 仿真器角色
DC-SCM	Datacenter Secure Control Module	OCP 定义的服务器控制模块标准
PLDM	Platform Level Data Model	DMTF 的平台级数据模型，用于固件更新与监控

缩写	全称	说明
SPDM	Security Protocol and Data Model	DMTF 的设备认证与安全会话协议
MCTP	Management Component Transport Protocol	管理组件传输协议
OCI	Open Container Initiative	此处用于承载签名固件包
PRoT / PFR	Platform Root of Trust / Platform Firmware Resilience	平台信任根 / 平台固件韧性
BMF	Boot Management Facility（Redfish）	通过 Redfish 触发固件启动的服务
DMI	Direct Media Interface	CPU 与 PCH 之间的链路，需训练